

EXPERIMENT-1

Student Name: Sejal
Branch: CSE-AIML
Semester: 5
Subject Name: ADBMS

UID: 24BAI70050
Section: 24AIT_KRG-1_B
Date of Performance: 11/7/26

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than \$20,000.
2. **"Average Salary"**: All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. **"High Salary"**: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table	
category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

```
SELECT 'LOW CATEORY' AS CATEGORY,  
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME <20000
```

UNION ALL

```
SELECT 'AVERAGE CATEORY' AS CATEGORY,  
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME >20000 AND INCOME <=50000
```

UNION ALL

```
SELECT 'HIGH CATEORY' AS CATEGORY,  
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME >50000;
```

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

(i)

```
SELECT E.*, D.*, S.*
FROM
EMPLOYEE AS E
JOIN
DEPARTMENT AS D
ON
E.DEPT_ID = D.DEPT_ID
JOIN
SALARIES AS S
ON
E.EMP_ID = S.EMP_ID;
```

(II)

```
UPDATE SALARIES
SET SALARY = SALARY + SALARY * 0.10
WHERE EMP_ID
IN
(
SELECT E.EMP_ID
FROM EMPLOYEE AS E
JOIN
DEPARTMENT AS D
ON
E.DEPT_ID = D.DEPT_ID
WHERE D.DEPT_NAME = 'HR'
)
```

(III) SELECT E.emp_id, E.name, D.dept_name, S.salary
FROM Employees E
JOIN Departments D

```

ON E.dept_id = D.dept_id
JOIN Salaries S
ON E.emp_id = S.emp_id
WHERE S.salary > (
SELECT AVG(salary)
FROM Salaries );

```

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`.

Every customer must choose **exactly 3 different toppings** to prepare a pizza.

Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.

The same topping **cannot appear more than once** in a pizza.

Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by **highest total cost**

If two pizzas have the same cost, sort them alphabetically.

```

SELECT
    CONCAT(T1.TOPPING_NAME, ',', T2.TOPPING_NAME, ',', T3.TOPPING_NAME) AS PIZZA,
    (T1.INGREDIENT_COST+T2.INGREDIENT_COST+T3.INGREDIENT_COST) AS TOTAL_COST
FROM PIZZA_TOPPINGS AS T1
JOIN
    PIZZA_TOPPINGS AS T2
ON
    T1.TOPPING_NAME < T2.TOPPING_NAME
JOIN
    PIZZA_TOPPINGS AS T3
ON
    T2.TOPPING_NAME < T3.TOPPING_NAME;

```

(D)

Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase. Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.
4. Same-day purchases must **not** be considered.
5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

```
SELECT DISTINCT p1.user_id
FROM Purchases p1
JOIN Purchases p2
ON p1.user_id = p2.user_id
WHERE p2.purchase_date > p1.purchase_date
AND p2.purchase_date <= p1.purchase_date + INTERVAL '7 days'
ORDER BY p1.user_id;
```